

MEMORIA SPRINT 3

Grupo 12.08 “LongNight”

Integrantes:

Jorge Carvajal Lara

Sergi Torres Albert

Brad Owen Valladares Quispe

Javier Perepérez Cardona

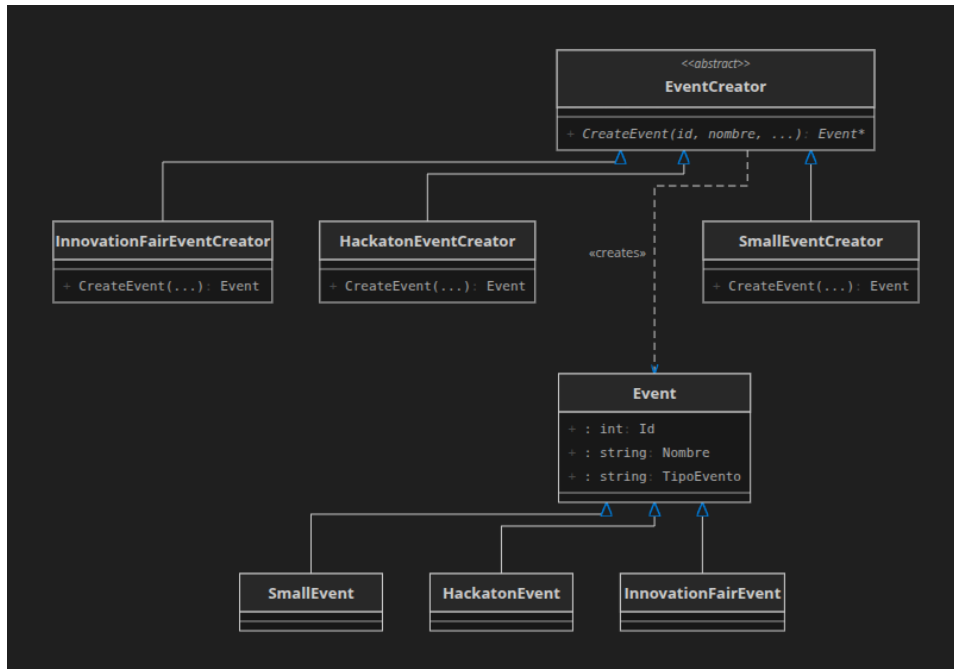
ÍNDICE DE LA MEMORIA

1. Patrones de Diseño Aplicados.....	3
1.1. Patrón de Diseño 1: Factory method	3
1.2. Patrón de Diseño 2: Repository method.....	8
1.3. Patrón de Diseño 3: Decorator.....	12
1.4. Patrón de Diseño 4: Strategy	16
1.5. Patrón de Diseño 5: Adapter.....	22
1.6. Patrón de Diseño 6: Filter.....	25
2. Refactoring.....	29
2.1. Refactorización 1: Extract Method.....	29
2.2. Refactorización 2: Move method.....	31
2.3. Refactorización 3: Parameterize method.....	32
2.4 Refactorización 4: Extract class	34
2.5 Refactorización 5: Inline class	35
3. Pruebas Unitarias (Testing).....	37
3.1. Ubicación de los Tests en GitHub.....	37
3.2. Estrategia y Cobertura de los Patrones.....	37
4. Uso y Documentación de IA Generativa - AI Usage Log	44
4.1. Herramientas usadas.....	44
4.2. Prompts clave.....	44
4.3. Salidas relevantes (resumen corto).....	46
4.4. Qué aceptamos y qué rechazamos (mínimo 3 ejemplos).....	47
4.5. Cómo lo verificamos.....	48
4.6. Resultado final / decisión humana.....	48

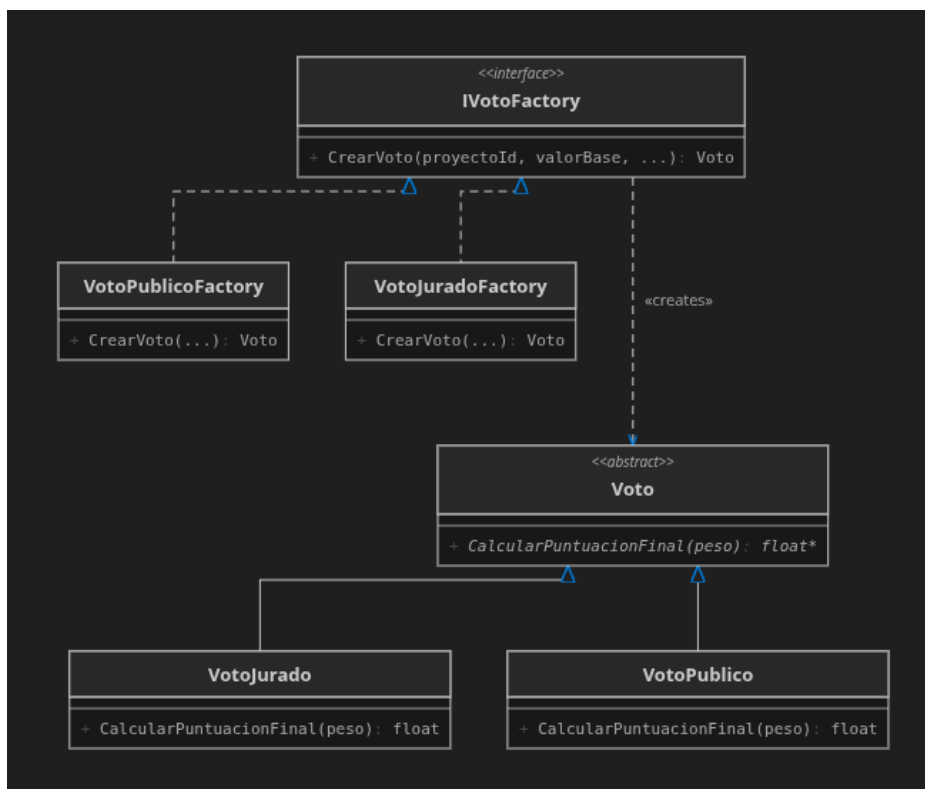
1. Patrones de Diseño Aplicados

1.1. Patrón de Diseño 1: Factory method

1.1.1 Estructura de Clases



El patrón se aplica también a la creación de votos, con una segunda jerarquía:



1.1.2 Código Representativo

Creador abstracto — EventCreator.cs

Clase abstracta que define el método factoría CreateEvent. Las subclasses concretas heredan de ella y proporcionan la implementación específica para cada tipo de evento.

```
public abstract class EventCreator
{
    public abstract Event CreateEvent(
        int id, string nombre, string descripcion,
        DateTime fechaini, DateTime fechafin, string estado,
        int idorganizador, List<Categoria> categorias,
        List<Baremo> baremos, int codEvento);
}
```

Creador concreto — HackatonEventCreator.cs

Una de las tres subclasses concretas del creador. Sobreescrive CreateEvent para instanciar un HackatonEvent con los parámetros recibidos. Las otras dos (SmallEventCreator, InnovationFairEventCreator) siguen la misma estructura.

```
public class HackatonEventCreator : EventCreator
{
    public override Event CreateEvent(
        int id, string nombre, string descripcion,
        DateTime fechaini, DateTime fechafin, string estado,
        int idorganizador, List<Categoria> categorias,
        List<Baremo> baremos, int codEvento)
    {
        return new HackatonEvent(
            id, nombre, descripcion, fechaini, fechafin,
            estado, idorganizador, categorias, baremos, codEvento);
    }
}
```

Interfaz Factory de Votos — IVotoFactory.cs

Segunda jerarquía del Factory Method aplicada a la creación de votos. Esta interfaz define el contrato CrearVoto que las fábricas concretas implementan según el rol del votante.

```
public interface IVotoFactory
{
    Voto CrearVoto(int proyectoId, float valorBase, int idCategoria,
                  int idCriterio, string? comentario, string? urlAudio,
                  int? idUsuario, string ipDispositivo);
}
```

Fábrica concreta — VotoJuradoFactory.cs

Fábrica concreta que crea instancias de VotoJurado. Nótese que asigna IdEvaluador = idUsuario, ya que un jurado es un votante identificado, a diferencia de VotoPublicoFactory donde el voto es anónimo.

```
public class VotoJuradoFactory : IVotoFactory
{
    public Voto CrearVoto(int proyectoId, float valorBase, int idCategoria,
                        int idCriterio, string? comentario, string? urlAudio,
                        int? idUsuario, string ipDispositivo)
    {
        return new VotoJurado
        {
            IdProyecto = proyectoId, Valor = valorBase,
            IdCategoria = idCategoria, IdCriterio = idCriterio,
            Comentario = comentario, UrlAudio = urlAudio,
            FechaVoto = DateTime.UtcNow,
            IdEvaluador = idUsuario, // El jurado SÍ usa el idUsuario
            IpDispositivo = ipDispositivo
        };
    }
}
```

Uso — selección de factory en CreateEventService.cs

El servicio cliente selecciona el creador concreto en runtime mediante un switch sobre el tipo de evento recibido del DTO, y luego invoca CreateEvent de forma polimórfica sin conocer la subclase concreta.

```
EventCreator creator = eventDto.TipoEvento switch
{
    "Hackaton" => new HackatonEventCreator(),
    "Evento Pequeño" => new SmallEventCreator(),
    "Feria" => new InnovationFairEventCreator(),
    _ => throw new ArgumentException($"Tipo de evento desconocido: {eventDto.TipoEvento}")
};

var NuevoEvento = creator.CreateEvent(0, eventDto.Nombre, ...);
```

Uso — selección de factory en VotoService.cs

En el servicio de votación, la selección de la fábrica se realiza consultando el rol del usuario en la base de datos. Si es jurado, se usa VotoJuradoFactory; en caso contrario, VotoPublicoFactory. El servicio trabaja siempre contra la interfaz IVotoFactory.

```
private async Task<IVotoFactory> ObtenerFactoryPorRolAsync(int eventId, int? idUsuario)
{
    if (!idUsuario.HasValue)
        return _votoPublicoFactory;

    var rol = await _votoRepository.ObtenerRolUsuarioEnEventoAsync(idUsuario.Value, eventId);
    return rol.Equals("Jurado", StringComparison.OrdinalIgnoreCase)
        ? _votoJuradoFactory
        : _votoPublicoFactory;
}
```

1.1.3 Justificación

Se eligió Factory Method para desacoplar la lógica de creación de los tipos concretos. El sistema necesita crear objetos polimórficos en dos contextos:

Eventos: Según TipoEvento (Hackaton, Feria, Evento Pequeño) se instancia la subclase correspondiente. Si se añade un nuevo tipo de evento, basta con crear un nuevo EventCreator y su producto sin modificar el servicio cliente.

Votos: Según el rol del usuario (Jurado vs Público) se instancia VotoJurado (con evaluador identificado) o VotoPublico (anónimo). VotoService trabaja contra IVotoFactory, ignorando los tipos concretos.

Esto respeta el principio Open/Closed: el sistema está abierto a extensión (nuevos tipos) pero cerrado a modificación del código existente.

Mejoras incorporadas en el Sprint 3:

Template Method en el Creator: CreateEvent() pasó de public abstract a protected abstract, y se añadió el método público PrepareEvent() que actúa como template method. El servicio cliente ahora llama a creator.PrepareEvent(...), no directamente al factory method.

```
// EventCreator.cs - ANTES
public abstract Event CreateEvent(int id, string nombre, ...);

// EventCreator.cs - DESPUÉS
public Event PrepareEvent(int id, string nombre, ...)
{ return CreateEvent(id, nombre, ...); } // punto de extensión
protected abstract Event CreateEvent(int id, string nombre, ...);
```

VotoFactoryBase — clase base abstracta compartida: VotoJuradoFactory y

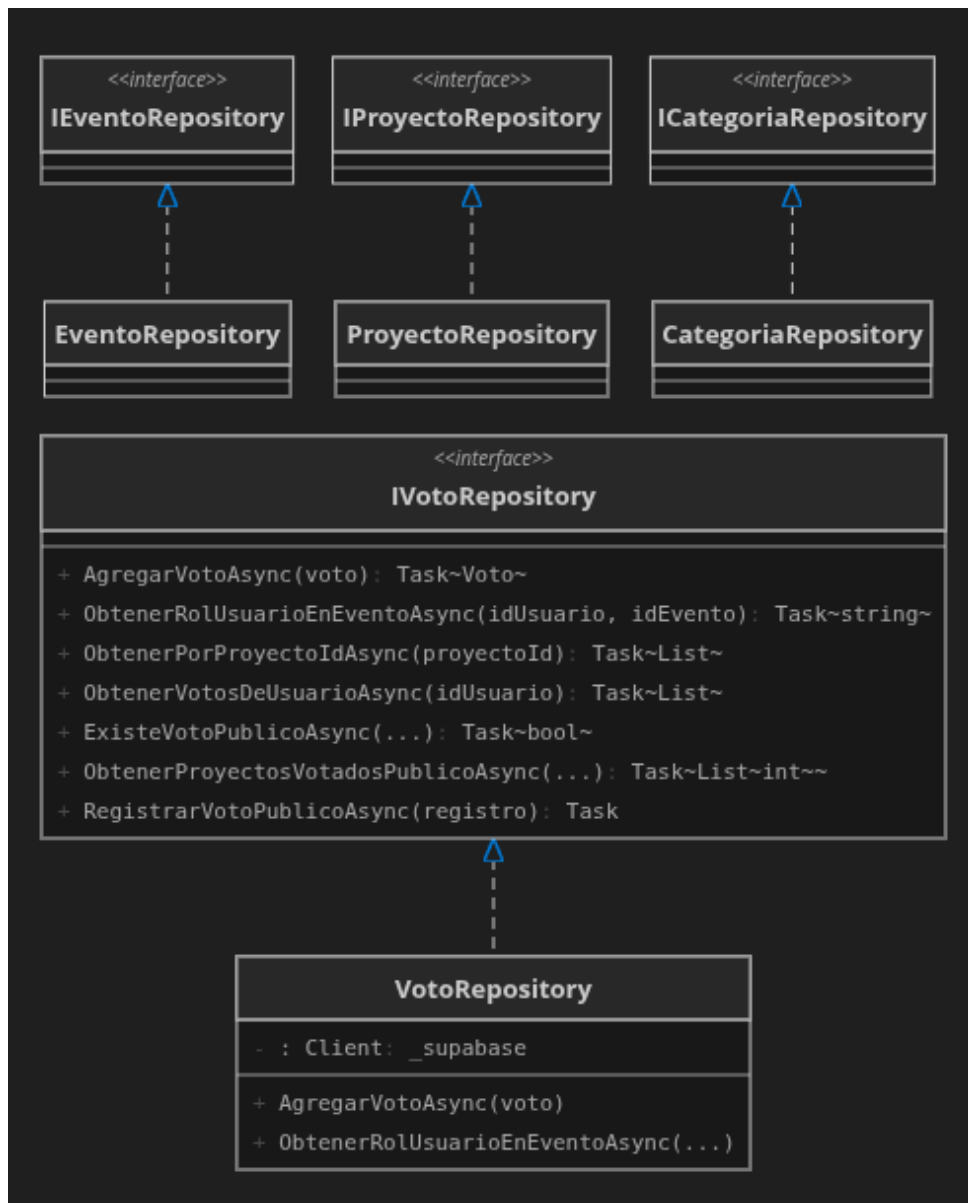
VotoPublicoFactory ahora extienden VotoFactoryBase : IVotoFactory. La base centraliza la asignación de FechaVoto = DateTime.UtcNow, eliminando la duplicación:

```
public abstract class VotoFactoryBase : IVotoFactory
{ public Voto CrearVoto(...) { var voto = CreateVoto(...); voto.FechaVoto =
DateTime.UtcNow; return voto; }

protected abstract Voto CreateVoto(...); }
```

1.2. Patrón de Diseño 2: Repository method

1.2.1 Estructura de Clases



1.2.2 Repositorios Implementados

Interfaz	Implementación
C# ICategoriaRepository	C# CategoriaRepository
C# IProyectoRepository	C# ProyectoRepository
C# IVotoRepository	C# VotoRepository
C# IUsuarioRepository	C# UsuarioRepository
C# IEventoRepository	C# EventoRepository
C# IEventoUsuarioRepository	C# EventoUsuarioRepository
C# IBaremoRepository	C# BaremoRepository
C# IInvitacionPendienteRepository	C# InvitacionPendienteRepository
C# IPremioRepository	C# PremioRepository
C# ISintesisRepository	C# SintesisRepository

1.2.3 Código Representativo

Interfaz del Repository — IVotoRepository.cs

Ejemplo representativo de la interfaz de un repositorio. Define las operaciones de acceso a datos para la entidad Voto como si fuera una colección en memoria (agregar, buscar, verificar existencia), ocultando completamente los detalles de Supabase/PostgreSQL.

```
public interface IVotoRepository
{
    Task<Voto> AgregarVotoAsync(Voto voto);
    Task<string?> ObtenerRolUsuarioEnEventoAsync(int idUsuario, int idEvento);
    Task<List<VotoPublico>> ObtenerPorProyectoIdAsync(int proyectoId);
    Task<List<VotoJurado>> ObtenerVotosDeUsuarioAsync(int idUsuario);
    Task<bool> ExisteVotoPublicoAsync(int idEvento, int idCategoria, string hash);
    Task<List<int>> ObtenerProyectosVotadosPublicoAsync(
        int idEvento, int idCategoria, string hash);
    Task RegistrarVotoPublicoAsync(RegistroVotoPublico registro);
    Task<List<Voto>> ObtenerVotosPorProyectoYCategoriaAsync(int proyectoId, int categoriaId);
}
```

Registro en DI — Program.cs

Registro de los 10 pares interfaz-implementación en el contenedor de dependencias. Los servicios de negocio reciben las interfaces por inyección de constructor, y el contenedor resuelve automáticamente la implementación concreta correspondiente.

```
builder.Services.AddScoped<ICategoriaRepository, CategoriaRepository>();
builder.Services.AddScoped<IProyectoRepository, ProyectoRepository>();
builder.Services.AddScoped<IVotoRepository, VotoRepository>();
builder.Services.AddScoped<IUsuarioRepository, UsuarioRepository>();
builder.Services.AddScoped<IEventoUsuarioRepository, EventoUsuarioRepository>();
builder.Services.AddScoped<IInvitacionPendienteRepository, InvitacionPendienteRepository>();
builder.Services.AddScoped<IEventoRepository, EventoRepository>();
builder.Services.AddScoped<IBaremoRepository, BaremoRepository>();
builder.Services.AddScoped<IPremioRepository, PremioRepository>();
builder.Services.AddScoped<ISintesisRepository, SintesisRepository>();
```

1.2.4 Justificación

Los 10 repositorios abstraen completamente el acceso a Supabase (PostgreSQL) detrás de interfaces. Los servicios de negocio dependen exclusivamente de las interfaces, no de la implementación concreta. Esto proporciona:

Desacoplamiento de la persistencia: Si se cambiara de Supabase a otra BD (Entity Framework, Dapper, MongoDB), solo habría que escribir nuevas implementaciones de los repositorios sin tocar la lógica de negocio.

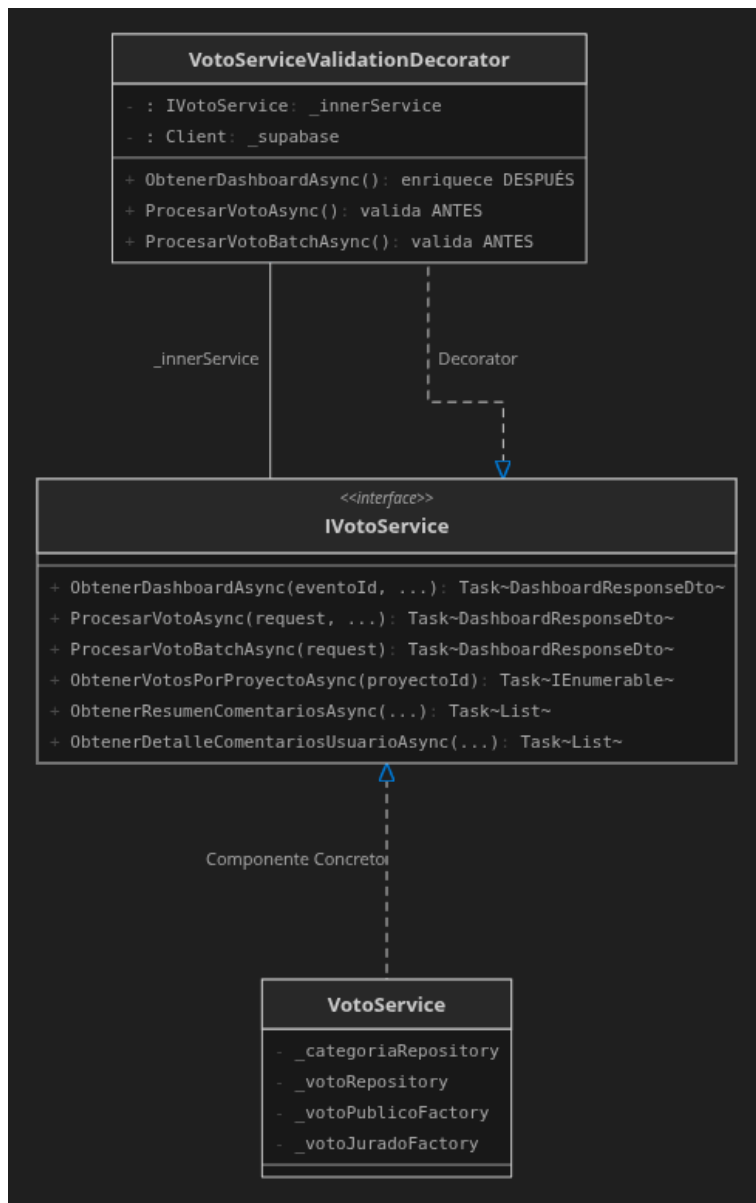
Testabilidad: El proyecto usa Moq extensivamente para mockear repositorios en tests unitarios (VotoServiceTests.cs, EventoServiceTests.cs, SintesisServiceTests.cs), lo cual sería imposible sin la abstracción por interfaces.

Principio de inversión de dependencias (DIP): Los módulos de alto nivel (servicios) no dependen de módulos de bajo nivel (acceso a datos), ambos dependen de abstracciones (interfaces).

Separación de responsabilidades: Cada repositorio se ocupa de una sola entidad del dominio, manteniendo cohesión y facilitando el mantenimiento.

1.3. Patrón de Diseño 3: Decorator

1.3.1 Estructura de Clases



1.3.2 Código Representativo

Interfaz componente — IVotoService.cs

Interfaz que actúa como Component del patrón Decorator. Define el contrato que tanto el servicio base (VotoService) como el decorador (VotoServiceValidationDecorator) implementan, garantizando la intercambiabilidad.

```
public interface IVotoService
{
    Task<DashboardResponseDto> ObtenerDashboardAsync(int eventoId, int? idUsuario = null,
        string? sessionId = null, string? identificadorHash = null);
    Task<DashboardResponseDto> ProcesarVotoAsync(VotoRequestDto request,
        int? idUsuario = null, string? sessionId = null);
    Task<DashboardResponseDto> ProcesarVotoBatchAsync(VotoBatchRequestDto request);
    Task<IEnumerable<VotoResponseDto>> ObtenerVotosPorProyectoAsync(int proyectoId);
    Task<List<TipoComentaristaDto>> ObtenerResumenComentariosAsync(int proyectoId, int categoriaId);
    Task<List<ComentarioDetalleDto>> ObtenerDetalleComentariosUsuarioAsync(
        int proyectoId, int categoriaId, string usuarioRef);
}
```

Decorator — VotoServiceValidationDecorator.cs

Clase decoradora que envuelve a IVotoService. Mantiene una referencia al servicio interno (_innerService) y añade comportamiento adicional: validación de comentarios antes de procesar votos, y enriquecimiento de datos después de obtener el dashboard. Los métodos sin lógica adicional simplemente delegan al servicio original.

```
public class VotoServiceValidationDecorator : IVotoService
{
    private readonly IVotoService _innerService; // referencia al componente decorado
    private readonly Supabase.Client _supabase;

    public VotoServiceValidationDecorator(IVotoService innerService, Supabase.Client supabase)
    {
        _innerService = innerService;
        _supabase = supabase;
    }

    // ENRIQUECE el resultado DESPUÉS de delegar
    public async Task<DashboardResponseDto> ObtenerDashboardAsync(...)
    {
        var dashboard = await _innerService.ObtenerDashboardAsync(eventoId, ...);
        var evento = ...; // consulta la configuración del evento
        if (evento != null)
            dashboardDto.ComentariosObligatorios = evento.ComentariosObligatorios;
        return dashboard;
    }

    // VALIDA ANTES de delegar (usa Strategy internamente)
    public async Task<DashboardResponseDto> ProcesarVotoAsync(VotoRequestDto request, ...)
    {
        var evento = ...;
        IComentarioValidationStrategy strategy = evento.ComentariosObligatorios
            ? new ComentarioObligatorioStrategy()
            : new ComentarioOpcionalStrategy();
        strategy.Validar(request.Comentario);
        // Si no hay excepción, delegamos al servicio original
        return await _innerService.ProcesarVotoAsync(request, idUsuario, sessionId);
    }

    // Métodos sin comportamiento adicional: DELEGAN directamente
    public Task<IEnumerable<VotoResponseDto>> ObtenerVotosPorProyectoAsync(int proyectoId)
        => _innerService.ObtenerVotosPorProyectoAsync(proyectoId);
}
```

Cableado en DI — Program.cs

Registro en el contenedor de inyección de dependencias. Se registra primero VotoService como tipo concreto, y luego IVotoService se resuelve creando el decorador que envuelve al servicio base. Así, cualquier clase que reciba IVotoService obtiene automáticamente la versión decorada, sin saberlo.

```
// Registro con Decorator para VotoService
builder.Services.AddScoped<VotoService>();
builder.Services.AddScoped<IVotoService>(provider =>
{
    var baseService = provider.GetRequiredService<VotoService>();
    var supabase = provider.GetRequiredService<Supabase.Client>();
    return new VotoServiceValidationDecorator(baseService, supabase);
});
```

1.3.3 Justificación

La validación y el enriquecimiento de datos de comentarios son responsabilidades transversales que no corresponden al núcleo de negocio transaccional del servicio.

Mapeo de Componentes del Libro de Gamma:

Componente (Component): IVotoService (interfaz común).

Componente Concreto (Concrete Component): VotoService (funcionalidad base de guardado y cálculo).

Decorador (Decorator): VotoServiceValidationDecorator (envoltura estructural).

Justificación de Adaptaciones y Reflexión sobre Desviaciones del Patrón Puro:

Desacoplamiento y Transparencia: El decorador se inyecta de forma transparente a través del contenedor de inversión de control (DI) en Program.cs. Los clientes (como los controladores ASP.NET Core) consumen IVotoService sin saber si están interactuando con el componente original o el decorador.

Reflexión sobre el "Decorator Puro":

Se reconoce que esta implementación presenta una desviación pragmática del Decorador puro clásico de Gamma. En un Decorador puro, una única responsabilidad añadida se aplica de manera uniforme y homogénea a todos los métodos de la interfaz (por ejemplo, logging de entrada/salida).

En nuestro caso, el decorador realiza acciones diferenciadas según la operación interceptada:

En `ProcesarVotoAsync` y `ProcesarVotoBatchAsync` añade validación antes de delegar (integrando internamente el patrón `Strategy`).

En `ObtenerDashboardAsync` añade enriquecimiento de metadatos después de delegar.

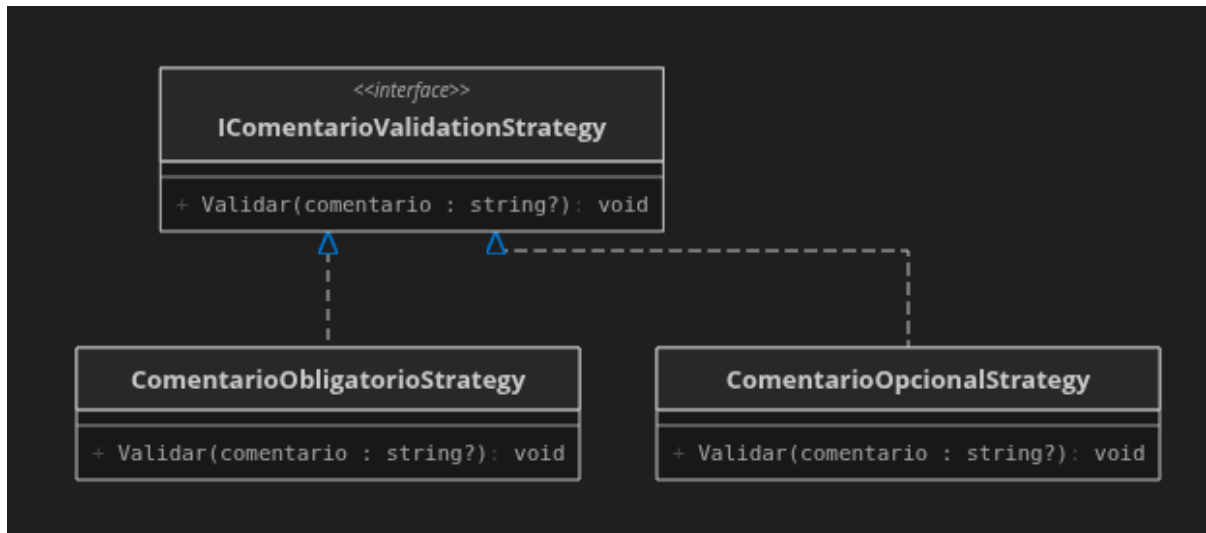
En el resto de operaciones (`ObtenerVotosPorProyectoAsync`, etc.), actúa como un mero `bypass` transparente.

Se adoptó este enfoque híbrido para evitar una explosión de clases decoradoras (lo que habría obligado a crear un `VotoServiceValidationDecorator` y un `VotoServiceEnrichmentDecorator` por separado, duplicando el código de delegación para los métodos `bypass` y sobrecargando el contenedor de dependencias). Esta decisión de diseño equilibra la pureza académica del libro de Gamma con la mantenibilidad real de una aplicación empresarial real en .NET.

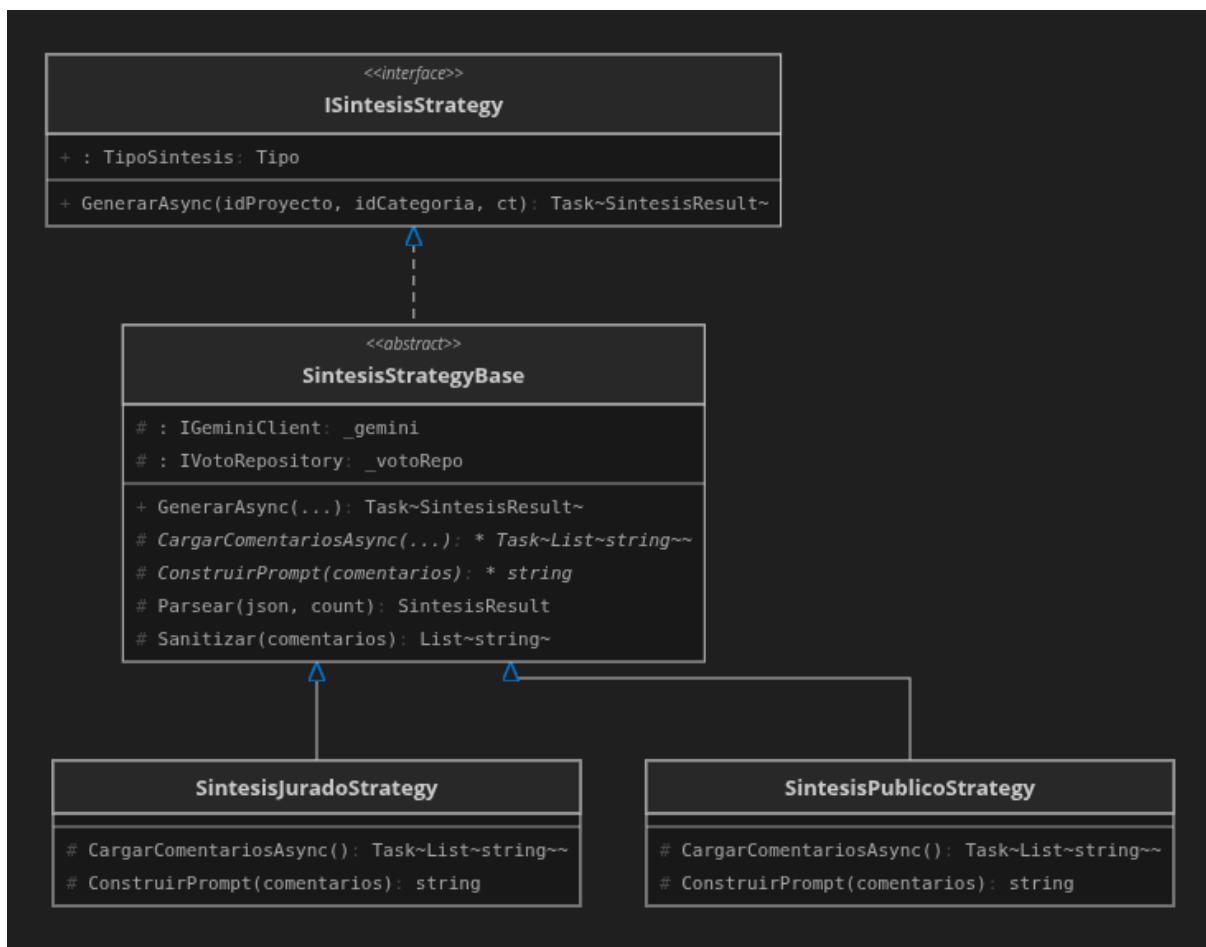
1.4. Patrón de Diseño 4: Strategy

1.4.1 Estructura de clases

Validación de comentarios:



Síntesis IA de comentarios:



1.4.2 Código Representativo

Interfaz Strategy — IComentarioValidationStrategy.cs

Interfaz que define el contrato común para todas las estrategias de validación de comentarios. Declara un único método Validar que cada estrategia concreta implementa con su propia lógica.

```
public interface IComentarioValidationStrategy
{
    void Validar(string? comentario);
}
```

Estrategia concreta (obligatorio) — ComentarioObligatorioStrategy.cs

Estrategia que lanza una excepción si el comentario está vacío o es nulo. Se selecciona cuando el evento tiene configurada la propiedad ComentariosObligatorios = true.

```
public class ComentarioObligatorioStrategy : IComentarioValidationStrategy
{
    public void Validar(string? comentario)
    {
        if (string.IsNullOrEmpty(comentario))
        {
            throw new ArgumentException(
                "El comentario es obligatorio para evaluar en este evento.");
        }
    }
}
```

Estrategia concreta (opcional) — ComentarioOpcionalStrategy.cs

Estrategia que no realiza ninguna validación, permitiendo que el comentario sea nulo o vacío. Se selecciona cuando ComentariosObligatorios = false.

```
public class ComentarioOpcionalStrategy : IComentarioValidationStrategy
{
    public void Validar(string? comentario)
    {
        // El comentario es opcional, no hacemos validación estricta
    }
}
```

Selección en runtime — VotoServiceValidationDecorator.cs

Momento en que el Contexto (el Decorator) selecciona la estrategia concreta en tiempo de ejecución, basándose en la configuración del evento. Después invoca Validar de forma polimórfica, sin conocer la implementación concreta.

```
// Aplicar Patrón Strategy para la validación
IComentarioValidationStrategy strategy = evento.ComentariosObligatorios
    ? new ComentarioObligatorioStrategy()
    : new ComentarioOpcionalStrategy();

strategy.Validar(request.Comentario);
```

Interfaz Strategy Síntesis — ISintesisStrategy.cs

Segunda aplicación del patrón Strategy en el dominio de síntesis IA. Define el contrato que deben cumplir las estrategias de generación de síntesis, con un método asíncrono GenerarAsync y una propiedad Tipo para identificar la audiencia.

```
public interface ISintesisStrategy
{
    TipoSintesis Tipo { get; }
    Task<SintesisResult> GenerarAsync(int idProyecto, int idCategoria,
                                     CancellationToken ct = default);
}
```

Template Method en la base — SintesisStrategyBase.cs

Clase abstracta que implementa el esqueleto del algoritmo de síntesis (Template Method). El método GenerarAsync orquesta los pasos en orden fijo: cargar comentarios → sanitizar → construir prompt → llamar a Gemini → parsear resultado. Los pasos marcados como abstract son los que cada subclase concreta redefine.

```
public async Task<SintesisResult> GenerarAsync(
    int idProyecto, int idCategoria, CancellationToken ct = default)
{
    var comentarios = await CargarComentariosAsync(idProyecto, idCategoria, ct); // abstracto
    var saneados = Sanitizar(comentarios); // concreto

    if (saneados.Count < 2)
        throw new SintesisInsuficienteException("Se requieren al menos 2 comentarios.");

    var prompt = ConstruirPrompt(saneados); // abstracto
    var json = await _gemini.GenerarJsonAsync(prompt, ct); // concreto
    return Parsear(json, saneados.Count); // virtual (hook)
}

protected abstract Task<List<string>> CargarComentariosAsync(int idP, int idC, CancellationToken ct);
protected abstract string ConstruirPrompt(List<string> comentarios);
```

Subclase concreta — SintesisJuradoStrategy.cs

Estrategia concreta para la audiencia de jurado. Sobreescrbe los pasos abstractos del Template Method: carga solo los comentarios de jurados (soloJurado: true) y construye un prompt específico orientado a feedback técnico profesional.

```
public class SintesisJuradoStrategy : SintesisStrategyBase
{
    public override TipoSintesis Tipo => TipoSintesis.Jurado;

    protected override Task<List<string>> CargarComentariosAsync(
        int idP, int idC, CancellationToken ct)
        => CargarComentariosVotoAsync(idP, idC, soloJurado: true, ct);

    protected override string ConstruirPrompt(List<string> comentarios)
    {
        var lista = FormatComentariosNumerados(comentarios);
        return $"Eres un asistente que sintetiza feedback de jurado...
Comentarios:
{lista}
Devuelve SOLO el JSON.";
    }
}
```

Factory de strategies — SintesisStrategyFactory.cs

Factory auxiliar que simplifica la selección de la estrategia de síntesis en runtime. Recibe un enumerado TipoSintesis y devuelve la instancia correspondiente, desacoplando al servicio cliente de las clases concretas.

```
public class SintesisStrategyFactory
{
    private readonly SintesisJuradoStrategy _jurado;
    private readonly SintesisPublicoStrategy _publico;

    public virtual ISintesisStrategy Crear(TipoSintesis tipo) => tipo switch
    {
        TipoSintesis.Jurado => _jurado,
        TipoSintesis.Publico => _publico,
        _ => throw new ArgumentOutOfRangeException(nameof(tipo))
    };
}
```

Interfaz ISintesisStrategyFactory (Sprint 3 — nueva):

Se añadió la interfaz ISintesisStrategyFactory para que la factory pueda ser mockeada en tests. SintesisComentariosService pasó a inyectar la interfaz en lugar de la clase concreta, respetando el principio de inversión de dependencias:

```
// ISintesisStrategyFactory.cs (nuevo)
public interface ISintesisStrategyFactory
{
    ISintesisStrategy Crear(TipoSintesis tipo);
}

// SintesisStrategyFactory.cs - ANTES
public class SintesisStrategyFactory    // sin interfaz, no mockeable
{
    public virtual ISintesisStrategy Crear(TipoSintesis tipo) { ... }
}

// SintesisStrategyFactory.cs - DESPUÉS
public class SintesisStrategyFactory : ISintesisStrategyFactory
{
    public ISintesisStrategy Crear(TipoSintesis tipo) { ... }    // sin virtual
}
```

1.4.3 Justificación

El patrón Strategy se aplica en dos escenarios independientes del dominio, cada uno con una motivación clara:

Mapeo de Componentes del Libro de Gamma:

Estrategia (Strategy): `IComentarioValidationStrategy` (para validación) e `ISintesisStrategy` (para síntesis).

Estrategias Concretas (ConcreteStrategies): `ComentarioObligatorioStrategy` / `ComentarioOpcionalStrategy` (en validación) y `SintesisJuradoStrategy` / `SintesisPublicoStrategy` (en síntesis).

Contexto (Context): `VotoServiceValidationDecorator` (en su rol de interceptor y orquestador) y `SintesisService` (en el módulo de IA).

Decisiones de Diseño y Adaptaciones:

Validación dinámica de comentarios: Cada evento configura dinámicamente si requiere o no comentarios para emitir un voto. Para evitar lógica de control estructurada (if/else repetitivos en los servicios), encapsulamos cada algoritmo en una estrategia dedicada.

Integración con Decorator: En lugar de acoplar la validación al servicio de negocio central, la estrategia es instanciada y ejecutada por el Decorador (`VotoServiceValidationDecorator`), que actúa como el Context del patrón. Esto aísla la validación del flujo transaccional del voto.

Síntesis de comentarios con IA (Strategy + Template Method): La generación de síntesis con Gemini requiere prompts y formas de cargar comentarios completamente distintas para jurados y público. Usamos Strategy para encapsular las diferencias del prompt y la carga de datos. Para evitar la duplicación de código en la carga y parseo comunes, lo combinamos de forma pragmática con el patrón Template Method en la clase abstracta `SintesisStrategyBase.cs` (definiendo el esqueleto de ejecución en `GenerarAsync` y delegando los pasos específicos a las estrategias concretas).

1.5. Patrón de Diseño 5: Adapter

1.5.1 Estructura de Clases



1.5.2 Código Representativo

Interfaz objetivo (Target) — IGeminiClient.cs

Interfaz de dominio (Target) que expone una API simplificada de un solo método (`GenerarJsonAsync`) para interactuar con un modelo de IA. Los servicios de la aplicación dependen exclusivamente de esta interfaz, sin conocer la API REST externa.

```
public interface IGeminiClient
{
    // Interfaz limpia del dominio
    Task<string> GenerarJsonAsync(string prompt, CancellationToken ct = default);

    // Modelo configurado (para auditoría)
    string ModelName { get; }
}
```

Adapter — GeminiClient.cs

Clase Adapter que traduce la interfaz simple del dominio a las llamadas HTTP complejas de la API REST de Google Gemini. Internamente construye los DTOs específicos de Gemini (GeminiRequest, GeminiContent, etc.), envía la petición POST, y extrae el texto de la respuesta JSON anidada, devolviendo al cliente un simple string.

```
public class GeminiClient : IGeminiClient
{
    private readonly HttpClient _http;
    private readonly string _apiKey;
    public string ModelName { get; }

    public GeminiClient(HttpClient http)
    {
        _http = http;
        _apiKey = Environment.GetEnvironmentVariable("GEMINI_API_KEY") ?? "";
        ModelName = Environment.GetEnvironmentVariable("GEMINI_MODEL") ?? "gemini-2.5-flash";
    }

    public async Task<string> GenerarJsonAsync(string prompt, CancellationToken ct = default)
    {
        // Construye la URL del API REST externo
        var url = $"https://generativelanguage.googleapis.com/v1beta/models/"
            + $"{ModelName}:generateContent?key={_apiKey}";

        // Construye el request body con DTOs específicos de Gemini (Adaptee)
        var request = new GeminiRequest
        {
            Contents = new List<GeminiContent>
            {
                new GeminiContent
                {
                    Role = "user",
                    Parts = new List<GeminiPart> { new GeminiPart { Text = prompt } }
                }
            },
            GenerationConfig = new GeminiGenerationConfig
            {
                ResponseMimeType = "application/json",
                ResponseSchema = SintesisSchema,
                Temperature = 0.4
            }
        };

        // Envía HTTP POST y traduce errores a excepciones del dominio
        var body = JsonSerializer.Serialize(request);
        using var content = new StringContent(body, Encoding.UTF8, "application/json");
        response = await _http.PostAsync(url, content, ct);

        // Extrae el texto del JSON anidado de respuesta de Gemini
        var text = parsed?.Candidates?.FirstOrDefault()
            ?.Content?.Parts?.FirstOrDefault()?.Text;

        return text;
    }
}
```

Registro en DI — Program.cs

Registro del adaptador en el contenedor de dependencias usando AddHttpClient, que además gestiona automáticamente el ciclo de vida del HttpClient subyacente.

```
builder.Services.AddHttpClient<IGeminiClient, GeminiClient>();
```

1.5.3 Justificación

La API REST de Gemini tiene una interfaz compleja: DTOs específicos (GeminiRequest, GeminiContent, GeminiGenerationConfig), respuestas anidadas (Candidates[0].Content.Parts[0].Text), gestión de errores HTTP con tipos específicos (rate limit, safety block, auth failure), y configuración de esquemas JSON para structured output. El Adapter encapsula toda esta complejidad detrás de una interfaz de dominio de un solo método: GenerarJsonAsync(prompt).

El directorio se llama literalmente Adapters/Gemini/, reflejando la intención del patrón. Beneficios clave:

Las estrategias de síntesis (patrón #2) solo dependen de IGeminiClient, no de la API de Google.

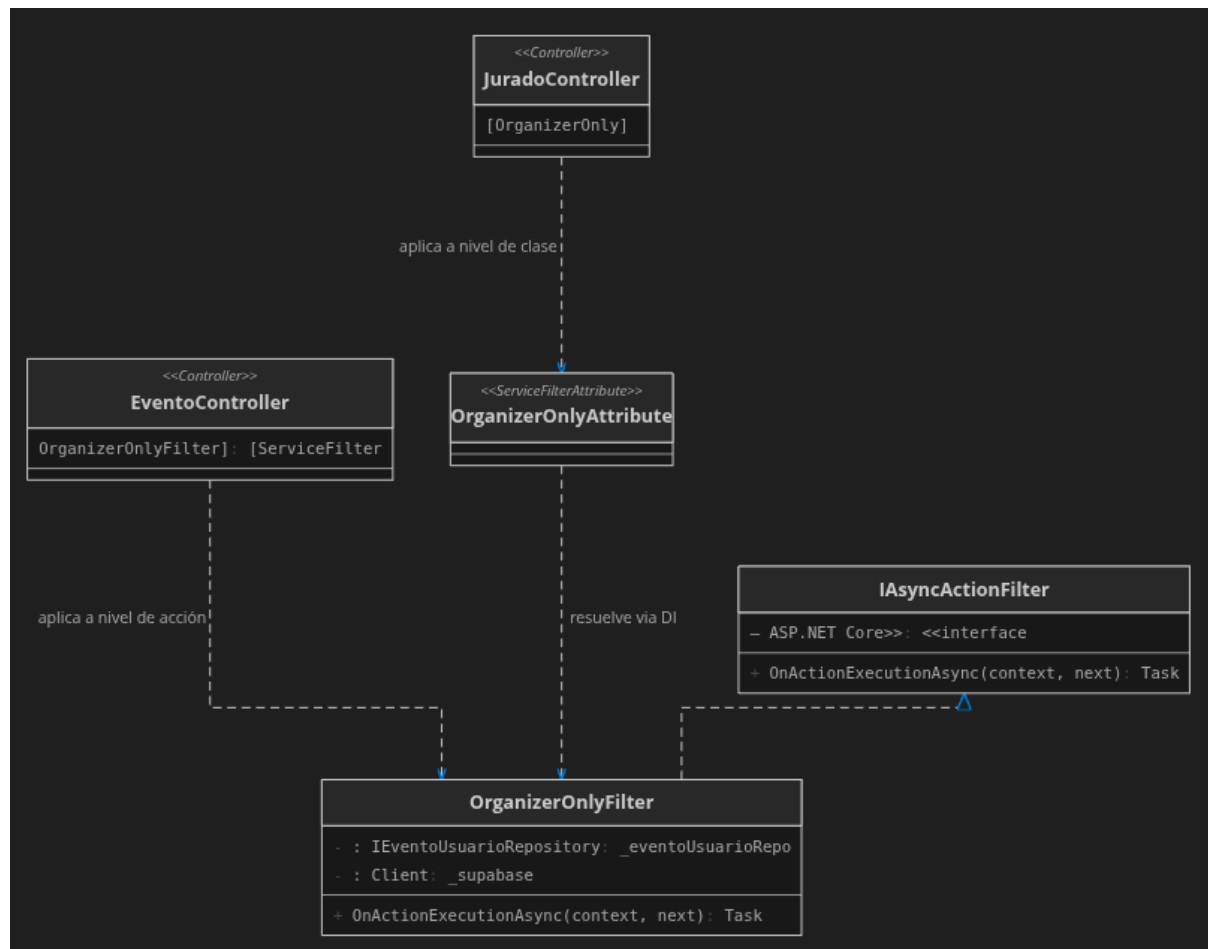
Es fácil sustituir Gemini por otro modelo de IA (OpenAI, Claude) implementando un nuevo IGeminiClient sin cambiar ningún servicio.

Los tests unitarios pueden mockear IGeminiClient trivialmente con Moq.

La traducción de errores HTTP a excepciones tipadas del dominio (SintesisGeminiException con SintesisGeminiErrorKind) centraliza el manejo de errores en un solo punto.

1.6. Patrón de Diseño 6: Filter

1.6.1 Estructura de Clases



1.6.2 Código Representativo

Atributo del filtro — OrganizerOnlyAttribute.cs

Atributo personalizado que extiende `ServiceFilterAttribute` para poder aplicar el filtro de forma declarativa con la anotación `[OrganizerOnly]`. Internamente resuelve **OrganizerOnlyFilter** a través de la inyección de dependencias.

```
public class OrganizerOnlyAttribute : ServiceFilterAttribute
{
    public OrganizerOnlyAttribute() : base(typeof(OrganizerOnlyFilter))
    {
    }
}
```

Implementación del filtro — OrganizerOnlyFilter.cs

Implementación central del filtro de autorización. En el Sprint 3 fue refactorizado: OrganizerOnlyFilter ya no inyecta Supabase.Client directamente para datos; ahora usa IUserarioRepository para consultar el usuario, manteniendo Supabase.Auth únicamente para validar el token de sesión (Auth SDK, no acceso a datos). Esto respeta la arquitectura Controllers → Services → Repositories → Supabase sin cortocircuitos.

```
public class OrganizerOnlyFilter : IAsyncActionFilter
{
    private readonly IEventoUsuarioRepository _eventoUsuarioRepo;
    private readonly Supabase.Client _supabase;

    public OrganizerOnlyFilter(IEventoUsuarioRepository eventoUsuarioRepo,
        Supabase.Client supabase)
    {
        _eventoUsuarioRepo = eventoUsuarioRepo;
        _supabase = supabase;
    }

    public async Task OnActionExecutionAsync(
        ActionExecutingContext context, ActionExecutionDelegate next)
    {
        // 1. Obtener el token de la cabecera Authorization
        string? token = context.HttpContext.Request.Headers["Authorization"]
            .ToString()?.Replace("Bearer ", "");

        if (string.IsNullOrEmpty(token))
        {
            context.Result = new UnauthorizedResult(); // 401
            return; // cortocircuito el pipeline
        }

        // 2. Validar el token con Supabase Auth
        var user = await _supabase.Auth.GetUser(token);
        if (user == null || string.IsNullOrEmpty(user.Email))
        {
            context.Result = new UnauthorizedResult(); // 401
            return;
        }

        // 3. Extraer idEvento de la ruta o del body
        int? idEvento = null;
        if (context.RouteData.Values.TryGetValue("idEvento", out var routeId))
        {
            if (int.TryParse(routeId.ToString(), out var parsedId))
            {
                idEvento = parsedId;
            }
        }

        if (!idEvento.HasValue)
        {
            context.Result = new BadRequestObjectResult("IdEvento no encontrado");
            return;
        }

        // 4. Verificar que el usuario tiene rol "Organizador" en el evento
        var dbUser = await _supabase.From<Usuario>()
            .Where(u => u.Email == user.Email).Single();
        var relacion = await _eventoUsuarioRepo.GetAsync(idEvento.Value, dbUser.Id);

        if (relacion == null || relacion.Rol != "Organizador")
        {
            context.Result = new ObjectResult(new {
                message = "No tienes permisos de organizador para este evento."
            }) { StatusCode = 403 }; // 403 Forbidden
            return;
        }

        // 5. Si todo OK, continuar al controller
        await next();
    }
}
```

Uso a nivel de clase — JuradoController.cs

Ejemplo de aplicación del filtro a nivel de clase mediante el atributo [OrganizerOnly]. Todas las acciones de JuradoController quedan protegidas automáticamente sin necesidad de repetir la lógica de autorización en cada método.

```
[ApiController]
[Route("api/[controller]")]
[OrganizerOnly] // - Filtro aplicado a TODAS las acciones del controller
public class JuradoController : ControllerBase
{
    ...
}
```

Uso a nivel de acción individual — EventoController.cs

Ejemplo de aplicación del filtro a nivel de acción individual usando [ServiceFilter]. Solo la acción EliminarEventoAsync queda protegida, mientras que las demás acciones de EventoController permanecen accesibles sin restricción de organizador.

```
[HttpDelete("{idEvento}")]
[ServiceFilter(typeof(OrganizerOnlyFilter))] // - Filtro en acción específica
public async Task<IActionResult> EliminarEventoAsync(int idEvento)
{
    await _eventoService.EliminarEventoAsync(idEvento);
    return NoContent();
}
```

Registro en DI — Program.cs

Registro del filtro en el contenedor de dependencias. Al registrarlo como Scoped, ASP.NET Core puede inyectar los repositorios y servicios que el filtro necesita para realizar las verificaciones de autorización.

```
builder.Services.AddScoped<OrganizerOnlyFilter>();
```

1.6.3 Justificación

La verificación de que un usuario es "Organizador" de un evento es una lógica de autorización transversal que se repite en múltiples controladores y acciones. Sin el filtro, habría que duplicar el código de validación de token + consulta a BD + verificación de rol en cada endpoint protegido.

El patrón Intercepting Filter resuelve esto:

OrganizerOnlyFilter encapsula toda la lógica de autorización en un solo lugar.

Se puede aplicar declarativamente con `[OrganizerOnly]` a nivel de clase (protege todas las acciones, como en `JuradoController`) o con `[ServiceFilter]` a nivel de acción individual (como `EliminarEventoAsync`).

El filtro puede cortocircuitar el pipeline devolviendo 401 (no autenticado) o 403 (no autorizado) sin que la petición llegue al controlador.

Al extender `ServiceFilterAttribute`, el filtro se resuelve por inyección de dependencias, permitiéndole acceder a repositorios y servicios como `IEventoUsuarioRepository`.

Si se necesitan nuevas políticas de acceso (ej. `JuradoOnlyFilter`), basta con crear un nuevo filtro sin modificar los existentes.

2. Refactoring

2.1. Refactorización 1: Extract Method

Antes: La página DashboardVotacionProyectos manejaba directamente la lógica algorítmica y estructural para dar formato a los objetos JSON requeridos por la API de votación. En este refactor usamos el handleConfirmarPublico como ejemplo, pero en handleConfirmarJurado también se aplica

```
try {
  const userIdRaw = localStorage.getItem('userId');
  const idUsuario = userIdRaw ? parseInt(userIdRaw) : null;
  const sessionId = localStorage.getItem('sessionId');
  const storedEventoId = localStorage.getItem('eventoId');
  const finalEventoId = eventoId || (storedEventoId ? parseInt(storedEventoId) : 0);

  const votoDto = {
    eventoId: finalEventoId,
    categoriaId: categoria.id,
    proyectoId: seleccionado.id,
    comentario: comentario,
    idUsuario: (idUsuario !== null && !Number.isNaN(idUsuario)) ? idUsuario : null,
    sessionId: sessionId || null,
    identificadorHash: fingerprint || undefined,
    valor: 1,
    idcriterio: null,
    idproyecto: seleccionado.id,
    idevaluador: idUsuario,
    idcategoria: categoria.id
  };
};
```

Después: El mapeo del objeto se delegó a una función externa e independiente (buildPublicVoteDto) en DashboardVotacionProyectosBuilders que encapsula la estructura de los datos del voto.

```

const votoDto = buildPublicVoteDto({
  eventoId: finalEventoId,
  categoriaId: categoria.id,
  proyectoId: seleccionado.id,
  comentario,
  idUsuario: idUsuario ?? undefined,
  sessionId,

  identificadorHash: fingerprint || undefined,
  valor: 1,
});

export function buildPublicVoteDto(params: {
  eventoId: number;
  categoriaId: number;
  proyectoId: number;
  comentario: string;
  idUsuario: number | undefined;
  sessionId: string | null;
  identificadorHash?: string;
  valor?: number;
}): any {
  const {
    eventoId,
    categoriaId,
    proyectoId,
    comentario,
    idUsuario,
    sessionId,
    identificadorHash,
    valor = 1,
  } = params;

  const idUsuarioUndefined = idUsuario ?? undefined;

  return {
    eventoId,
    categoriaId,
    proyectoId,
    comentario,
    idUsuario: idUsuarioUndefined,
    sessionId,
    identificadorHash: identificadorHash || undefined,
    valor,
    idcriterio: null,
  };
}

```

Beneficio: Se limpia el flujo visual del componente y se obtiene una función pura, aislada y fácilmente testeable mediante pruebas unitarias.

2.2. Refactorización 2: Move method

Antes: Después del refactor del extract method se iba a mantener las llamadas al localStorage en el archivo DashboardVotacionProyectos.

```
const userIdRaw = localStorage.getItem('userId');
const idUsuario = userIdRaw ? parseInt(userIdRaw) : null;
const sessionId = localStorage.getItem('sessionId');
const storedEventoId = localStorage.getItem('eventoId');
const finalEventoId = eventoId || (storedEventoId ? parseInt(storedEventoId) : 0);
```

Después: Se traslado la lectura de datos del localStorage a otro archivo, dashboardVotacionProyectosBuilders donde tiene más sentido que este, de esta forma se limita más las funciones de DashboardVotacionProyectos.

```
export function getLocalVotingContext(params: {
  eventoIdFromContext: number | null | undefined;
  categoriaId: number;
}): LocalVotingContext {
  const { eventoIdFromContext } = params;

  const userIdRaw = localStorage.getItem('userId');
  const idUsuario = userIdRaw ? parseInt(userIdRaw) : null;

  const sessionId = localStorage.getItem('sessionId');

  const storedEventoId = localStorage.getItem('eventoId');
  const finalEventoId = eventoIdFromContext ?? (storedEventoId ? parseInt(storedEventoId) : 0);

  return {
    idUsuario: idUsuario !== null && Number.isNaN(idUsuario) ? null : idUsuario,
    sessionId: sessionId || null,
    finalEventoId,
  };
}
```

Beneficio: Se respeta el principio de responsabilidad única, reduciendo el acoplamiento del componente visual y facilitando el mantenimiento del código a largo plazo.

2.3. Refactorización 3: Parameterize method

Antes: En `DashboardVotacionProyectos` se repetía la misma estructura de control de concurrencia y manejo de excepciones (try/finally) en múltiples manejadores de eventos de confirmación.

```
submitLock.current = true;
setSubmitting(true);

} finally {
  submitLock.current = false;
  setSubmitting(false);
}
```

Después: Se unificó el comportamiento duplicado en la función genérica `withSubmitGuard`, la cual recibe la lógica variable como un parámetro ejecutable (action).

```
export async function withSubmitGuard(params: {
  submitLock: { current: boolean };
  setSubmitting: Dispatch<SetStateAction<boolean>>;
  action: () => Promise<void>;
}): Promise<void> {
  const { submitLock, setSubmitting, action } = params;
  if (submitLock.current) return;

  submitLock.current = true;
  setSubmitting(true);
  try {
    await action();
  } finally {
    submitLock.current = false;
    setSubmitting(false);
  }
}
```

```

await withSubmitGuard({
  submitLock,
  setSubmitting,
  action: async () => {
    const { finalEventoId, idUsuario, sessionId } = getLocalVotingContext({
      eventoIdFromContext: eventoId,
      categoriaId: categoria.id,
    });

    const votoDto = buildPublicVoteDto({
      eventoId: finalEventoId,
      categoriaId: categoria.id,
      proyectoId: seleccionado.id,
      comentario,
      idUsuario: idUsuario ?? undefined,
      sessionId,

      identificadorHash: fingerprint || undefined,
      valor: 1,
    });

    await enviarVoto(votoDto as any);
    alVolver();
  },
});

```

Beneficio: Se elimina la duplicación de código de infraestructura (DRY) y se centraliza la política de seguridad contra doble envío en un solo punto ajustable. Esto servirá si en el futuro hay más roles.

2.4 Refactorización 4: Extract class

Antes: En ComentarioCualitativoService teníamos en el constructor el el Supabase.Client y en varios métodos teníamos queries para acceder a base de datos. Un pequeño ejemplo :

```
public class ComentarioCualitativoService : IComentarioCualitativoService
{
    private readonly Supabase.Client _supabase;

    public ComentarioCualitativoService(Supabase.Client supabase)
    {
        _supabase = supabase;
    }

    public async Task<List<ComentarioCualitativo>> GetComentariosPorVotacion(long idVotacion)
    {
        var response = await _supabase
            .From<ComentarioCualitativo>()
            .Where(x => x.IdVotacion == idVotacion)
            .Get();

        return response.Models;
    }
}
```

Después: Se utiliza un nuevo repositorio ComentarioCualitativoRepository de forma que mantenemos separa la capa de persistencia.

```
private readonly Repositories.IComentarioCualitativoRepository _comentariosRepo;

0 references
public ComentarioCualitativoService(Repositories.IComentarioCualitativoRepository comentariosRepo)
{
    _comentariosRepo = comentariosRepo;
}

1 reference
public async Task<List<ComentarioCualitativo>> GetComentariosPorVotacion(long idVotacion)
{
    return await _comentariosRepo.GetComentariosPorVotacion(idVotacion);
}

4 references
public async Task<ComentarioCualitativo?> CreateComentarioAsync(
    long idVotacion,
    string comentarioTexto)
{
    var comentario = new ComentarioCualitativo
    {
        IdVotacion = idVotacion,
        Comentario = comentarioTexto,
        Fecha = DateTime.UtcNow
    };

    return await _comentariosRepo.CreateComentarioAsync(comentario);
}
```

Beneficio: Al crear el repositorio necesario, estamos extrayendo la lógica de persistencia a una nueva clase independiente.

2.5 Refactorización 5: Inline class

Antes: La clase `CategoriasService` solo tenía un método que no es tan usado.

```
public CategoriaService(Client supabase)
{
    _supabase = supabase;
}

public async Task<Categoria> CreateAsync(Categoria categoria)
{
    try
    {
        var response = await _supabase
            .From<Categoria>()
            .Insert(categoria);

        if (response?.Models == null || response.Models.Count == 0)
        {
            throw new Exception("No se pudo crear la categoría");
        }

        return response.Models[0];
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al crear la categoría: {ex.Message}", ex);
    }
}
```

Después: Se insertó en la clase createEventService donde tiene sentido y se utiliza.

```
public class CreateEventService : ICreateEventService
{
    private List<Categoria> MapearCategorias(List<CreateCategoriaDto> dtos)
    {
        return categorias;
    }
}

2 references
public async Task<Categoria> CreateAsync(Categoria categoria)
{
    try
    {
        var response = await _supabaseClient
            .From<Categoria>()
            .Insert(categoria);

        if (response?.Models == null || response.Models.Count == 0)
        {
            throw new Exception("No se pudo crear la categoría");
        }

        return response.Models[0];
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al crear la categoría: {ex.Message}", ex);
    }
}
```

Beneficio: Nos ahorramos crear dos clases : un Service y su interfaz. Además la funcionalidad de la clase ahora está mejor en createEventService donde hace más sentido.

3. Pruebas Unitarias (Testing)

3.1. Ubicación de los Tests en GitHub

El repositorio organiza las pruebas unitarias de manera paralela al código fuente. Para el backend, se ha creado el proyecto Votify.API.Tests, donde cada controlador, servicio o repositorio (*.cs) tiene su correspondiente clase de prueba (*.Tests.cs). En el frontend, los tests de cada componente se encuentran en una subcarpeta __tests__ contigua al componente, o en /src/tests/unit, promoviendo una estructura modular y de fácil localización por parte de los desarrolladores.

3.2. Estrategia y Cobertura de los Patrones

Resultado de ejecución actualizado (27 tests, todos en verde):

```
dotnet test Backend/Votify.sln
Iniciando la ejecución de pruebas, espere...
1 archivos de prueba en total coincidieron con el patrón especificado.
Correctas! - Con error: 0, Superado: 27, Omitido: 0, Total: 27, Duración: 678 ms
```

Cobertura de patrones en los tests:

Test	Patrón cubierto
SintesisStrategyTests.cs	Strategy + Template Method
SintesisServiceTests.cs	Strategy (ISintesisStrategyFactory)
VotoServiceTests.cs	Factory Method (IVotoFactory)
VotoServiceRefactorTests.cs	Decorator (delegación)
ComentarioValidationTests.cs	Strategy (IComentarioValidationStrategy)
EventoServiceTests.cs	Repository Pattern
CategoriasControllerTests.cs	Repository Pattern

Directorio en GitHub: Backend/Votify.Tests/

La estrategia de testing se ha centrado en el aislamiento y la velocidad, aplicando la Pirámide de Testing para priorizar las pruebas unitarias.

3.3. Detalle de los Tests Implementados y Evidencias

CASO 1: Prueba del Repositorio de Proyectos (Obtener por ID)

Objetivo: Verificar que el repositorio de proyectos recupera correctamente un proyecto existente desde la base de datos cuando se le proporciona un identificador válido, asegurando que el mapeo de datos funciona correctamente.

Descripción del test:

Este test unitario valida el método `ObtenerPorIdAsync` del repositorio de proyectos. Mediante mocks del cliente Supabase, se simula la existencia de un proyecto en la base de datos, verificando que el repositorio:

Recibe correctamente un ID de proyecto como parámetro de entrada

Ejecuta la consulta a la base de datos a través del cliente Supabase

Recupera el registro correspondiente de la tabla proyecto

Mapea correctamente los datos de la base de datos a la entidad de dominio Proyecto

Devuelve el objeto Proyecto completo con todas sus propiedades (Id, Nombre, Descripción, etc.)

Maneja adecuadamente la ausencia de resultados (retorna null)

Justificación del valor del test:

Este test es crítico porque:

Validación del mapeo objeto-relacional: Asegura que la capa de acceso a datos transforma correctamente los registros de Supabase en objetos de dominio sin pérdida de información ni errores de tipo, garantizando que los datos que circulan por el sistema mantienen su integridad.

Operación fundamental del sistema: La consulta "buscar proyecto por ID" es una de las operaciones más frecuentes en la aplicación. Se utiliza en la visualización de detalles del proyecto, edición de proyectos, sistema de votación, cálculo de rankings y generación de feedback. Una falla aquí afectaría a múltiples módulos del sistema.

Base para pruebas de integridad referencial: Validar que la recuperación del proyecto mantiene todas sus relaciones (evento asociado, categoría asignada, participantes del equipo) es esencial para el correcto funcionamiento de los módulos de votación y ranking, ya que estas relaciones determinan qué proyectos compiten entre sí y quiénes pueden votarlos.

Aislamiento de la base de datos: Permite validar la lógica del repositorio sin necesidad de una base de datos real, lo que garantiza pruebas rápidas, repetibles y sin dependencias externas, facilitando la detección temprana de errores durante el desarrollo.

Evidencia:

```
PS C:\Users\bpere\source\repos\TrabajoPSW-DDS-Javier\Backend\Votify.API> dotnet
test ../Votify.Tests/Votify.Tests.csproj --filter "ObtenerPorId"

Restauración completada (0,4s)

Votify.API realizado correctamente (0,3s) → bin\Debug\net9.0\Votify.API.dll

Votify.Tests correcto con 1 advertencias (3,1s) → C:\Users\bpere\source\repos
\TrabajoPSW-DDS-Javier\Backend\Votify.Tests\bin\Debug\net9.0\Votify.Tests.dll

C:\Users\bpere\source\repos\TrabajoPSW-DDS-Javier\Backend\Votify.Tests
\ObtenerPorId.cs(32,47): warning CS8602: Desreferencia de una referencia posiblemente
NULL.

[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.14)

[xUnit.net 00:00:00.77] Discovering: Votify.Tests

[xUnit.net 00:00:00.80] Discovered: Votify.Tests

[xUnit.net 00:00:00.81] Starting: Votify.Tests

[xUnit.net 00:00:00.92] Finished: Votify.Tests

Votify.Tests prueba realizado correctamente (2,4s)

Resumen de pruebas: total: 1; con errores: 0; correcto: 1; omitido: 0; duración: 2,4 s

Compilación correcto con 1 advertencias en 6,6s
```

Interpretación:

El test ha pasado correctamente. Verifica que el repositorio recupera un proyecto existente desde Supabase cuando se le proporciona un ID válido. El mapeo objeto-relacional entre la base de datos y la entidad de dominio funciona correctamente.

Caso 2: Prueba del Adaptador Gemini (IA)

Objetivo: Verificar que el adaptador Gemini maneja correctamente los diferentes escenarios de comunicación con la API de IA de Google, incluyendo errores de autenticación, límites de tasa, timeouts y respuestas exitosas.

Descripción del test:

Este test unitario valida el adaptador que integra el sistema con la API de Gemini de Google para la generación automática de feedback (fortalezas, mejoras, sentimiento y resumen de comentarios). Mediante mocks del cliente HTTP, se simulan los distintos comportamientos de la API externa, verificando que el adaptador:

Detecta y rechaza peticiones sin API key configurada

Procesa correctamente respuestas JSON válidas con el esquema requerido

Maneja errores de rate limit (429 Too Many Requests)

Gestiona timeouts en la comunicación

Trata respuestas vacías o malformadas

Justificación del valor del test:

Este test es crítico porque:

Aislamiento de dependencias externas: Permite validar la lógica del adaptador sin realizar llamadas reales a la API de Google, lo que garantiza pruebas rápidas, repetibles y sin coste asociado.

Cobertura de casos de error: Asegura que el sistema responde adecuadamente ante fallos de red, autenticación, rate limiting o respuestas malformadas, previniendo comportamientos inesperados en producción y garantizando una experiencia de usuario robusta.

Validación del esquema estructurado: Comprueba que el adaptador respeta el esquema JSON requerido para la síntesis de feedback, asegurando que los datos generados por IA se integran correctamente con el resto del sistema.

Evidencia:

```
PS C:\Users\bpere\source\repos\TrabajoPSW-DDS-Javier\Backend\Votify.API> dotnet
test ../Votify.Tests/Votify.Tests.csproj --filter "FullyQualifiedName~GeminiClient"

Restauración completada (0,3s)

Votify.API realizado correctamente (0,3s) → bin\Debug\net9.0\Votify.API.dll

Votify.Tests realizado correctamente (0,1s) → C:\Users\bpere\source\repos\TrabajoPSW-
-DDS-Javier\Backend\Votify.Tests\bin\Debug\net9.0\Votify.Tests.dll

[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.14)
[xUnit.net 00:00:00.04] Discovering: Votify.Tests
[xUnit.net 00:00:00.06] Discovered: Votify.Tests
[xUnit.net 00:00:00.06] Starting: Votify.Tests
[xUnit.net 00:00:00.30] Finished: Votify.Tests

Votify.Tests prueba realizado correctamente (0,9s)

Resumen de pruebas: total: 6; con errores: 0;
```

Interpretación:

Los 6 tests han pasado correctamente. El adaptador Gemini maneja adecuadamente:

- Respuestas exitosas de la API
- Ausencia de API key (autenticación)
- Límite de peticiones (rate limit)
- Timeout de conexión
- Respuestas vacías

CASO 3: Prueba del Filtro de Autorización (OrganizerOnlyFilter)

Objetivo: Verificar que el filtro OrganizerOnlyFilter restringe correctamente el acceso a los endpoints de organizador, validando que solo los usuarios autenticados con rol de "Organizador" en el evento específico puedan acceder a los recursos protegidos.

Descripción del test:

Este test unitario valida el comportamiento del filtro de acción personalizado que protege los endpoints sensibles del organizador. Mediante la simulación de diferentes escenarios de petición, se verifica que el filtro:

Rechaza peticiones sin token de autenticación con código 401 Unauthorized

Rechaza peticiones con token vacío con código 401 Unauthorized

Permite la continuación de la petición cuando el token es válido y el IdEvento existe

Valida la presencia del parámetro IdEvento en la ruta, devolviendo 400 Bad Request si falta

Bloquea el acceso con código 403 Forbidden cuando el usuario no tiene rol de organizador

Justificación del valor del test: Este test es crítico porque:

Seguridad del sistema: El filtro es la principal barrera de seguridad para endpoints sensibles como la configuración de votaciones, gestión de categorías y control de tiempos de votación. Una falla aquí podría permitir que usuarios no autorizados modifiquen eventos ajenos, comprometiendo la integridad de las votaciones.

Prevención de escalado de privilegios: Asegura que un participante común (rol "Público") no pueda ejecutar acciones reservadas exclusivamente para organizadores, manteniendo la integridad del modelo de permisos y garantizando que solo los creadores del evento puedan modificar su configuración.

Validación multifactorial: El filtro combina múltiples verificaciones (existencia del token, validez del token, presencia del IdEvento, relación de usuario con el evento, rol específico). Cada una de estas validaciones es esencial y debe probarse individualmente para garantizar que no existen brechas de seguridad.

Experiencia de usuario: Los códigos de error diferenciados (401 vs 403 vs 400) permiten al cliente de la API entender qué ha fallado específicamente, facilitando la depuración y mejorando la experiencia del desarrollador al integrarse con la API.

Evidencia:

```
PS C:\Users\bpere\source\repos\TrabajoPSW-DDS-Javier\Backend> dotnet test
Votify.Tests\Votify.Tests.csproj --filter "FullyQualifiedName~FilterTests"
```

Restauración completada (0,4s)

Votify.API realizado correctamente (0,3s) → Votify.API\bin\Debug\net9.0\Votify.API.dll

Votify.Tests correcto con 1 advertencias (0,4s) → Votify.Tests\bin\Debug\net9.0\Votify.Tests.dll

C:\Users\bpere\source\repos\TrabajoPSW-DDS-Javier\Backend\Votify.Tests\ObtenerPorId.cs(32,47): warning CS8602: Desreferencia de una referencia posiblemente NULL.

[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.14)

[xUnit.net 00:00:00.78] Discovering: Votify.Tests

[xUnit.net 00:00:00.80] Discovered: Votify.Tests

[xUnit.net 00:00:00.81] Starting: Votify.Tests

[xUnit.net 00:00:00.92] Finished: Votify.Tests

Votify.Tests prueba realizado correctamente (1,7s)

Resumen de pruebas: total: 5; con errores: 0; correcto: 5; omitido: 0; duración: 1,6 s

Compilación correcto con 1 advertencias en 3,1s

Interpretación:

El filtro de autorización **protege correctamente los endpoints sensibles**. Implementa una barrera de seguridad completa verificando: token de autenticación, presencia de parámetros y permisos de organizador.

4. Uso y Documentación de IA Generativa - AI Usage Log

4.1. Herramientas usadas

- **Herramienta/modelo:** ChatGPT-4o, GitHub Copilot, Gemini, Antigravity, Gemini CLI, Claude Code
- **Para qué se uso:**
 - Planificación y refinamiento de requisitos.
 - Arquitectura y diseño del sistema (Backend y Frontend).
 - Generación de lógica de negocio y código.

4.2. Prompts clave

- **Prompt 1 (UT 3387 — Síntesis IA):** Actúa como desarrollador .NET 9. Necesito implementar el patrón Strategy para síntesis de comentarios con Gemini. Hay dos tipos: jurado (foco técnico) y público (foco en experiencia). La respuesta debe ser un JSON estricto con fortalezas[], mejoras[], sentimiento y resumen_general. El cliente HTTP ya existe en GeminiClient.cs. Muéstrame cómo estructurar SintesisJuradoStrategy y SintesisPublicoStrategy heredando de una base común, y cómo pasar el JSON schema a Gemini para forzar la salida estructurada."
- **Prompt 2 (UT 3786 — Ranking de categorías):** Tengo un endpoint /categorias/{id}/ranking que devuelve proyectos ordenados por puntuación. Necesito un componente React + TypeScript RankingCategoria.tsx que muestre posición, nombre del proyecto y puntuación, con medallas visuales para el top 3 y usando el color del evento desde EventContext.userColor. El componente se monta dentro del dashboard del participante.
- **Prompt 3: (UT 4101 — Aceleradores de teclado):** Necesito atajos de teclado en la SPA React: Ctrl+K para abrir búsqueda global, Escape para cerrar modales y panels, y navegación con flechas en listas de proyectos. No hay ningún sistema de gestión de teclas actualmente. ¿Qué patrón conviene más, un hook useKeyboardShortcuts centralizado o useEffect local por componente? Justifica la decisión y muestra el hook si lo recomiendas.
- **Prompt 4 (UT 4100 — Ayuda contextual):** Genera el contenido de ayuda contextual para las 3 vistas principales de Votify: (1) Panel del Organizador - crear categorías y activar votación, (2) Panel del Jurado - evaluar proyectos por criterios con peso; (3) Panel del Participante - ver ranking y feedback generado por IA. Máximo 3 bullets por vista, tono instructivo y directo, en español. Se mostrará como tooltip/panel de ayuda dentro de la interfaz.

- **Prompt 5:** "Actúa como evaluador académico de patrones de diseño. Tengo la memoria de un sprint con 6 patrones implementados (Factory Method, Repository, Decorator, Strategy, Adapter, Filter) y necesito identificar inconsistencias entre lo documentado y el código real del Sprint 3. Concretamente: (1) ¿el patrón Factory Method refleja correctamente el uso de PrepareEvent() como template method y la clase base VotoFactoryBase? (2) ¿El Decorator documenta la eliminación de Supabase.Client directo? (3) ¿La sección de Strategy menciona ISintesisStrategyFactory como interfaz mockeable? Dame una lista y sugiere cómo redactar cada corrección manteniendo el estilo técnico del documento original. Además revisa los demás puntos de la memoria, en concreto el punto 2 y el 3, para que estén correctamente escritos y sugiere cambios a realizar"
- **Prompt 6:** "Actúa como revisor de código senior y auditor de calidad de software. Necesito hacer una revisión final exhaustiva de la entrega del Sprint 3 de Votify antes del cierre. Revisa que todos los patrones estructurales y de comportamiento estén correctamente implementados y conectados entre el backend (.NET 9) y el frontend (React/TypeScript). Comprueba la calidad de los comentarios (que aporten valor y no sean redundantes), que no queden líneas muertas, bloques TODO pendientes, y valida que tanto el frontend como el backend cumplan los criterios de aceptación generales del sprint. Señala riesgos potenciales."

4.3. Salidas relevantes (resumen corto)

- **Prompt 1:** Propuso `SintesisStrategyBase` abstracta con `BuildPrompt()` y `GetSchema()` a sobrescribir; las estrategias concretas devuelven prompts distintos y el mismo schema JSON. `SintesisStrategyFactory` selecciona la estrategia según el tipo recibido.
- **Prompt 2:** Componente con `useEffect` + `fetch` en `mount`, array mapeado a filas con iconos de medalla (o número), y clase CSS dinámica ligada a `userColor`. Propuso también un `skeleton loader` mientras carga.
- **Prompt 3:** Recomendó un hook centralizado `useKeyboardShortcuts(shortcuts: Record<string, () => void>)` registrado en el componente raíz, con limpieza del listener en el return del `useEffect`. Incluyó ejemplos para `Ctrl+K` y `Escape`.
- **Prompt 4:** Generó bloques de texto para las 3 vistas con bullets concisos; algunos textos usaban terminología genérica que no coincidía con la nomenclatura interna del proyecto (por ej., "evaluación" en lugar de "voto de jurado")
- **Prompt 5:** La IA identificó correctamente los 5 gaps principales: la justificación de `Factory Method` no mencionaba `PrepareEvent()` ni `VotoFactoryBase`; el código representativo del `Decorator` seguía mostrando la dependencia directa a `Supabase.Client` ya eliminada; `Strategy` no documentaba `ISintesisStrategyFactory` ni su impacto en los tests; `Adapter` no reflejaba la extracción de `GeminiExceptions.cs`; y `Filter` describía `OrganizerOnlyFilter` con acceso directo a datos en lugar del repositorio. Para cada problema propuso un párrafo de redacción en tono técnico consistente con el resto del documento. Además de que sugirió cambios a realizar en los puntos 2 y 3 y una mejora de redacción.
- **Prompt 6:** Confirmó la correcta integración general de patrones entre el backend y el frontend, validando el tipado de contratos API. Identificó 4 comentarios redundantes en la capa de servicios del backend que repetían la firma del método en lenguaje natural, y alertó sobre un archivo de pruebas en el frontend (`App.test.tsx`) que arrojaba un error debido a que el sistema de atajos de teclado interceptaba eventos globales de teclado sin un mock adecuado en el entorno de testing.

4.4. Qué aceptamos y qué rechazamos (mínimo 3 ejemplos)

- **Aceptado:** La estructura de SintesisStrategyBase con BuildPrompt() y GetSchema() abstractos encaja limpiamente con el patrón Strategy ya presente en el proyecto y facilita añadir nuevos tipos de síntesis en el futuro.
- **Aceptado:** El hook useKeyboardShortcuts centralizado, en lugar de múltiples useEffect dispersos, evita duplicar listeners y centraliza la gestión de conflictos entre atajos.
- **Aceptado parcialmente:** El skeleton loader del componente de ranking se simplificó a un spinner estándar del proyecto en lugar del skeleton personalizado propuesto, con el objetivo de mantener coherencia visual.
- **Rechazado:** El uso de emojis de medalla directamente en JSX (🏆 🥈 🥉) se sustituyó por los iconos SVG del design system del proyecto para cumplir las guías de diseño.
- **Corregido:** Los textos de ayuda contextual (Prompt 4) utilizaban “evaluación” y “puntuación” de forma genérica; se reescribieron para emplear la terminología exacta del dominio (“voto de jurado”, “baremo” y “criterio con peso”).
- **Aceptado (Prompt 5):** Se aceptó la estructura de análisis gap-por-gap y la redacción propuesta para las secciones de Decorator y Adapter, que encajaban directamente con el estilo existente. Además de cambios y mejoras de las redacciones de los puntos 2 y 3 ya que se explicaban de manera mas técnica y específica.
- **Rechazado (Prompt 5):** Se rechazó la sugerencia de reescribir completa la justificación de Factory Method: la IA proponía eliminar la explicación de Open/Closed para dar protagonismo a las nuevas mejoras, cuando lo correcto era añadirlas al final sin tocar lo ya validado.
- **Aceptado (Prompt 6):** Se aceptó la depuración de los comentarios innecesarios del backend, mejorando la mantenibilidad del código bajo la premisa de que "el buen código se documenta a sí mismo".
- **Rechazado (Prompt 6):** La IA recomendó implementar una biblioteca de terceros de manejo robusto de atajos para mitigar riesgos futuros, pero fue rechazado por el equipo para evitar sobrecargar las dependencias de la SPA y mantener la solución nativa y ligera.

4.5. Cómo lo verificamos

- **[X] Tests unitarios** — SintesisServiceTests.cs y SintesisStrategyTests.cs: 6/6 pruebas en verde, EventoServiceTests.cs: 9/9 pruebas en verde.
- **[X] Compilación backend** — ejecución de dotnet build completada sin errores tras integrar las estrategias.
- **[X] Compilación TypeScript** — ejecución de npx tsc --noEmit completada sin errores nuevos en los componentes añadidos.
- **[X] Consulta a la documentación oficial** — el schema de salida estructurada de Gemini fue validado contra la documentación oficial de Google AI Studio.
- **[X] Revisión por pares** — los textos de ayuda contextual fueron revisados y corregidos por el equipo antes de su integración.

4.6. Resultado final / decisión humana

El equipo adoptó el patrón Strategy para la síntesis IA tal como fue propuesto, incorporando los ajustes necesarios de nomenclatura del dominio. El hook useKeyboardShortcuts se integró en el componente raíz con el objetivo de centralizar la gestión de atajos de teclado.

Los textos de ayuda contextual fueron corregidos manualmente antes de su integración definitiva en la aplicación. Asimismo, las decisiones arquitectónicas relacionadas con la síntesis IA quedaron documentadas en ADR-009.

Por otra parte, el ranking de categorías se incorporó al dashboard del participante como una nueva pestaña dentro del hub, decisión registrada y justificada en ADR-010.

La IA fue muy útil como auditor de consistencia entre documentación y código (Prompt 5), un rol donde su capacidad de detectar omisiones en texto estructurado es alta. Sin embargo, todas las correcciones pasaron por revisión manual antes de integrarse: el criterio de qué añadir, qué conservar y qué redactar de nuevo fue siempre una decisión del equipo. El documento final refleja el estado real del código, no la interpretación de la IA sobre él.

Finalmente, en cuanto a la revisión global de la entrega (Prompt 6), delegar en la IA el rol de revisor senior de código funcionó como una excelente red de seguridad. Nos permitió interceptar un test roto en el frontend y pulir la calidad estética del código eliminando comentarios innecesarios antes del envío final, además de mejorar la implementación para seguir el libro de manera más correcta. Las decisiones críticas sobre arquitectura y alcance se mantuvieron firmes bajo criterio humano para evitar la introducción de dependencias de terceros innecesarias a última hora.